Enhancing Small Language Models in Agentic Pipelines with Prompt Engineering and Fine-Tuning

Yi-Hsiang Chen, Feng-Rui Liao, Jun-Cheng Zhang, Zheng-Hao Huang, Chin-Yu Sun*

Dept. of CSIE, National Taipei University of Technology

{t111590037, t111590019, t111590045, t111590018, cysun}@ntut.edu.tw

Abstract

Recent advances in large language models have shown remarkable performance in code generation, often achieving state-of-the-art results across benchmarks. However, their substantial computational requirements limit accessibility, reproducibility, and scalability outside specialized cloud infrastructures. This gap between capability and deployability highlights the need for approaches that balance efficiency with performance. To address this challenge, we introduce strategies for enhancing the coding capabilities of small-scale language models integrated into agentic pipelines suitable for consumer-grade hardware. Results show that our two-step pipeline achieves performance competitive with significantly larger models while maintaining efficiency suitable for consumer PCs. These findings indicate that well-designed workflows can mitigate the limitations of smaller models and extend their usability. More broadly, this study highlights that the combining of prompt engineering and fine-tuning within agentic pipelines offers an effective and resource-efficient approach for building capable coding assistants.

Agentic Pipeline

We posit that generating high-quality answers requires high-quality questions. To this end, we design an agent pipeline that simultaneously improves both the quality of user-posed questions and the model's ability to respond effectively to them. The pipeline consists of two stages.

1. **Question Reformulate Model**
   Input: Python function signature.
   Output: Reformulated, well-structured question.
2. **Code Generation Model**
   Input: Reformulated question.
   Output: Complete code.

Experimental results demonstrate that this pipeline significantly enhances the language model's code generation performance. Figure 3. illustrates the proposed agent pipeline.

Evaluation

We evaluate our proposed pipeline using the HumanEval benchmark released by OpenAI, which consists of 164 incomplete Python code snippets. Our pipeline completes these snippets and is then evaluated using the provided test cases. Figure 1 shows the scores obtained before and after fine-tuning. Figure 2 compares our results with state-of-the-art (SOTA) models.

Fine tune

We selected two open-weight small language models, Llama-3.2-1B-Instruct and Qwen2.5-Coder-1.5B-Instruct, as our fine-tuning targets. The fine-tuning process utilized the OpenCodeInstruct dataset, a synthetic dataset generated by Qwen2.5-Coder-32B-Instruct and open-sourced by NVIDIA. OpenCodeInstruct contains five million instruction–code pairs.

For the training environment, we utilize eight NVIDIA H100-80GB GPUs with model parallelism applied. The training was conducted with an initial learning rate of 2e-5, a batch size of 768, and for 3 epochs. The fine-tuning of Llama-3.2-1B-Instruct and Qwen2.5-Coder-1.5B-Instruct required approximately 120 GPU hours and 200 GPU hours, respectively.

For the training templates, we followed the base models' original instruction formats. In the question reformulation model, the user input contained the function signature, while the assistant output contained the problem description, enabling question reconstruction. In contrast, the code generation model used the reformulated question as the user input and the assistant output included both the function signature and the corresponding code implementation.

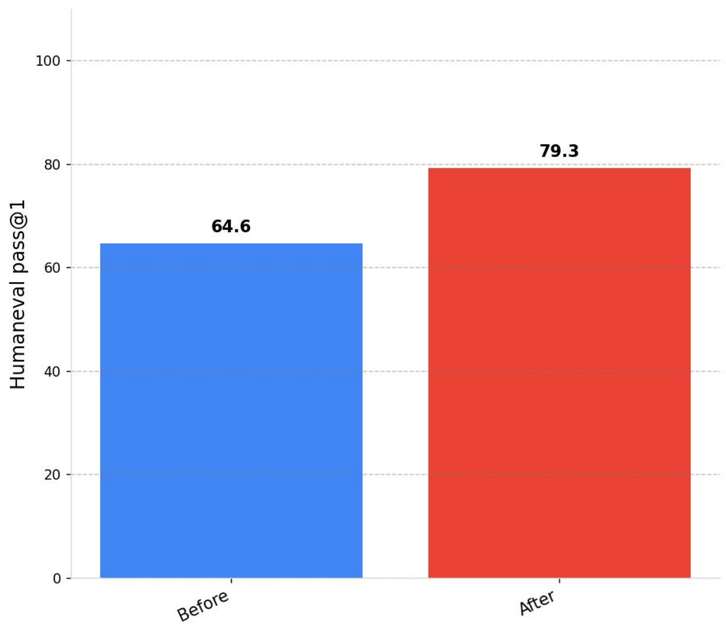


Figure 1. With/Without fine-tune compare

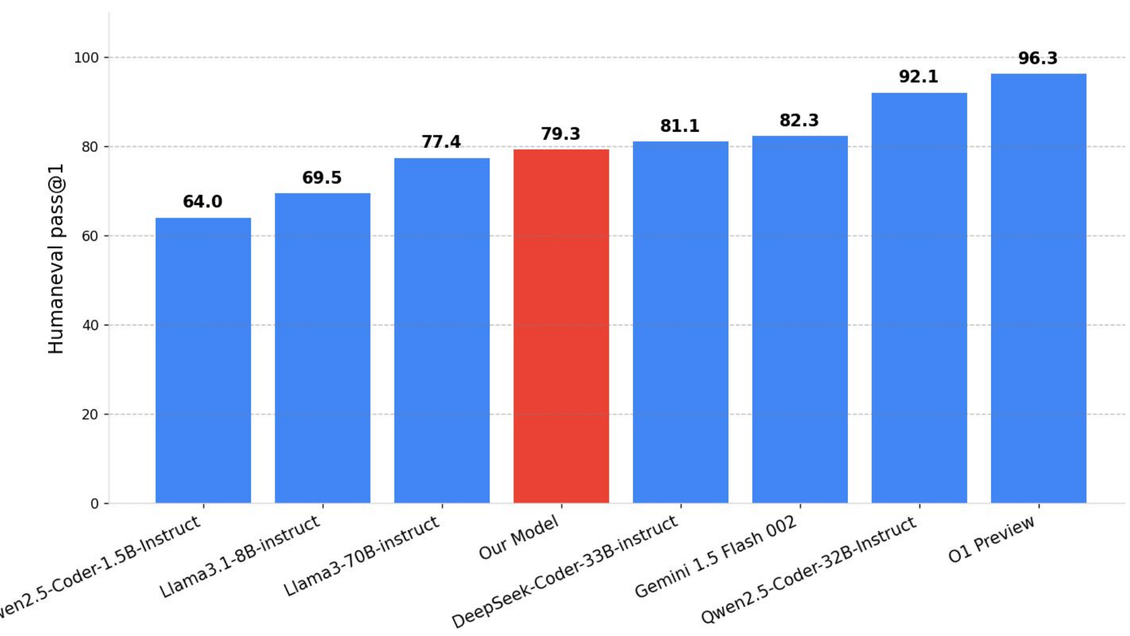


Figure 2. Compare with SOTA model

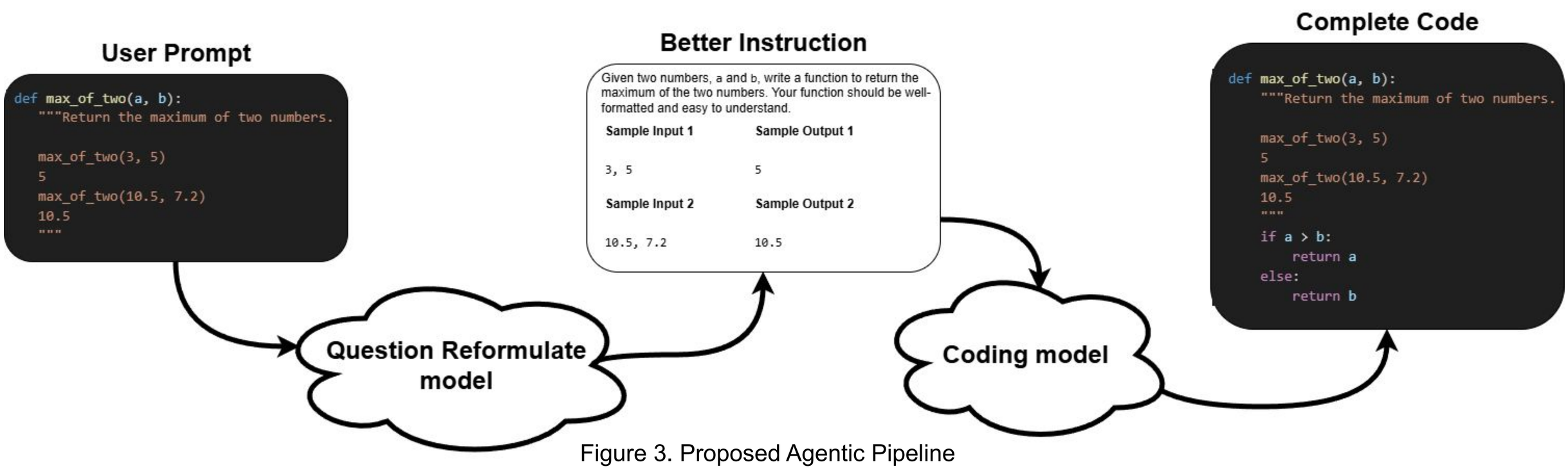


Figure 3. Proposed Agentic Pipeline